\WOOLF/

Applied Software Project Report

By

Keshari Nandan Prasad

A Master's Project Report submitted to Scaler Neovarsity - Woolf in partial fulfillment of the requirements for the degree of Master of Science in Computer Science

OCTOBER, 2026

SCALER

Scaler Mentee Email ID : kesharionemail@gmail.com

Thesis Supervisor : Naman Bhalla

Date of Submission :12/10/2026

© The project report of Keshari Nandan Prasad is approved, and it is acceptable in quality and form for publication electronically

Certification

I confirm that I have overseen / reviewed this applied project and, in my judgment, it adheres to the appropriate standards of academic presentation. I believe it satisfactorily meets the criteria, in terms of both quality and breadth, to serve as an applied project report for the attainment of Master of Science in Computer Science degree. This applied project report has been submitted to Woolf and is deemed sufficient to fulfill the prerequisites for the Master of Science in Computer Science degree.

Naman Bhalla

……

Project Guide / Supervisor

DECLARATION

I confirm that this project report, submitted to fulfill the requirements for the Master of Science in Computer Science degree, completed by me from 10 Sep 2026 to 11 Oct 2026, is the result of my own individual endeavor. The Project has been made on my own under the guidance of my supervisor with proper acknowledgement and without plagiarism. Any contributions from external sources or individuals, including the use of AI tools, are appropriately acknowledged through citation. By making this declaration, I acknowledge that any violation of this statement constitutes academic misconduct. I understand that such misconduct may lead to expulsion from the program and/or disqualification from receiving the degree.

Keshari Nandan Prasad

Keshari Nandan Prasad **Date: 12 Oct 2026**

ACKNOWLEDGMENT

I am grateful first to my wife, Akanksha, who made room for the evenings and weekends this degree asked of us, and to my mother, whose encouragement has followed every stage of my education. I thank my thesis supervisor, Naman Bhalla, and the Scaler Neovarsity instructors for a capstone that rewarded engineering trade-offs rather than a checklist. The work itself was self-motivated: I wanted to show, with measurements rather than slogans, that a flash sale can refuse oversell under a million attempts. That resolve, and Akanksha's patience when the laptop stayed open late, carried this report to completion.

Table of Contents

List of Tables 6
List of Figures 8
Applied Software Project 10
Abstract 10
1. Project Description 12
2. Requirement Gathering 16
3. Class Diagrams 22
4. Database Schema Design 28
5. Feature Development Process 35
6. Deployment Flow 45
7. Technologies Used 50
8. Conclusion 57
References 60

List of Tables

(To be written sequentially as they appear in the text)

Table No.	Title	Page No.
2.01	Users and actors of the flash sale platform	16
2.02	Use case detail for UC-02 Claim a unit	17
2.03	Functional requirements	18
2.04	Non-functional requirements	19
2.05	Feature set with priority	20
3.01	Domain classes, responsibilities and protected invariants	23
4.01	app_user	29
4.02	flash_sale	29
4.03	inventory	29
4.04	reservation	30
4.05	customer_order	30
4.06	outbox_event	30
4.07	Foreign key relationships	31
4.08	Cardinality between entities	31
4.09	Indexes	32
4.10	Redis keys per sale	33
5.01	Request elements of the claim endpoint	36

5.02	Response outcomes of the claim endpoint	36
5.03	Benchmark environment and parameters	42
5.04	Baseline strategy, one million attempts	42
5.05	Optimised strategy, one million attempts	43
5.06	Reconciler expiry query before and after the partial index	43
6.01	Subnet layout	45
6.02	Security group rules	46
6.03	AWS components and their role	48
6.04	Indicative monthly cost, small production footprint	49
7.01	Technology summary	50
8.01	Limitations and suggested improvements	58

List of Figures

(List of Images, Graphs, Charts sequentially as they appear in the text)

Figure No.	Title	Page No.
1.01	Project development process	14
1.02	High-level architecture of the flash sale platform	14
2.01	Use case diagram	17
2.02	Sale and reservation state machines	18
3.01	Domain class diagram	23
3.02	Application services, ports and infrastructure adapters	24
3.03	Sequence diagram of the claim feature including alternative flows	26
4.01	Entity-relationship diagram	28
5.01	Request flow through security, controller, service and infrastructure layers	38
5.02	Two concurrent claims for the last unit resolved by Redis's single command thread	40
5.03	Baseline claim path compared with the optimised claim path	41
6.01	AWS deployment architecture	45
6.02	Continuous integration and deployment pipeline	48
7.01	Layered structure of the application on the Spring Boot runtime	51
7.02	Kafka topic, partitions, consumer group and dead-letter topic	54

Applied Software Project

Abstract

Limited-stock product launches, ticket releases and festival discount events share one uncomfortable property: a very large number of buyers arrive within the same few seconds and compete for a small, fixed number of units. A conventional e-commerce checkout, in which every request locks and updates an inventory row in a relational database, degrades badly under this pattern. Requests queue on a single hot row, response times climb, buyers retry, and the retries deepen the queue. Worse, when the system is under this stress it is also most likely to sell the same unit twice.

This project designs, implements and measures a **High-Concurrency Flash Sale Platform**: a backend reservation service that admits purchase attempts for a single sale at in-memory speed while keeping a relational database as the durable record of truth. The system is built with Java 25 and Spring Boot 4, PostgreSQL 16, Redis 7 and Apache Kafka. An atomic Redis Lua script acts as the admission gate, checking remaining stock, the per-user limit and an idempotency key in one indivisible step and issuing a short-lived hold. Accepted claims are persisted together with a transactional outbox row, relayed to Kafka, and fulfilled asynchronously by an order worker that charges the buyer's saved payment method and creates the order idempotently. A reconciler expires abandoned holds and continuously verifies the stock invariant.

The work is evaluated by driving one million purchase attempts from a k6 load generator against a sale of one thousand units, first through a baseline path that performs all work inside the request transaction, then through the optimised path. The comparison reports throughput, latency percentiles, and the oversell count, which must be zero in both cases. The report also presents the low-level design, the database schema, a full trace of the claim feature through the MVC layers, and a production deployment design on AWS using Elastic Beanstalk, RDS, ElastiCache and MSK inside a private VPC.

The techniques demonstrated apply directly to ticketing, limited-edition retail, seat and slot booking, and any system where correctness under a burst matters more than average-case throughput.

1. Project Description

The problem being solved

A flash sale is a short, scheduled sale of a limited quantity of a product at an attractive price. Retailers use it to launch products, clear inventory or drive traffic; ticketing platforms face the same shape of demand when a popular event goes on sale. What makes the problem interesting from an engineering point of view is not the volume of successful orders, which is bounded by stock and therefore small, but the volume of attempts. A sale of one thousand units can easily attract hundreds of thousands of attempts in its first minute, and almost all of them must be rejected quickly, fairly and without damaging the system.

Three failure modes appear in naive implementations. The first is **overselling**: two requests read the same stock value, both decide there is a unit left, and both write an order. The second is **collapse under load**: every request contends for the same database row, lock wait time becomes the dominant cost, connection pools are exhausted, and the whole application, including pages unrelated to the sale, becomes unavailable. The third is **unfairness and abuse**: a single buyer scripting many parallel requests can take a disproportionate share of the stock, and duplicate submissions caused by client retries produce duplicate orders.

This project treats a flash sale as a **reservation problem** rather than a shopping-cart problem. The service's job is to decide, for each attempt, whether that buyer gets a unit, and to do so with the following guarantees: the number of confirmed orders never exceeds the initial stock; a buyer never receives more than the per-user limit regardless of how many requests they send; a retried request produces the same answer as the original; and a unit that was reserved but never paid for returns to the pool so the sale does not under-sell either.

Objectives

The objectives were fixed at the start of the module and drove every design decision:

1. Build a backend service in Java and Spring Boot that accepts purchase claims for a scheduled, limited-stock sale and turns accepted claims into paid orders.

2. Guarantee the four invariants above under genuine concurrency, and prove it with automated concurrency tests, not by inspection.
3. Keep the relational database as the source of truth while removing it from the hot path of admission decisions.
4. Make the system observable and safe to operate: explicit failure behaviour when a dependency is unavailable, metrics for every decision the gate makes, and a reconciliation job that detects drift.
5. Measure, rather than assert, the effect of the architecture. Run the same one-million-attempt workload against a baseline implementation and the optimised implementation and report the numbers.
6. Design a deployment on AWS that a small team could actually run, with private networking, managed data services and a repeatable release process.

Scope and boundaries

The platform deliberately models one sale at a time with one SKU and a per-user limit of one unit, because that is the case in which contention is highest and the design is easiest to reason about and verify. Multi-SKU catalogues, carts, shipping and tax are out of scope; they are ordinary CRUD concerns that add pages of code without adding to the problem being studied. Payment is modelled through a `PaymentGateway` port with a simulated adapter that has configurable latency and failure rate. This mirrors how flash sales are run in practice, where buyers pre-register a payment method so that the purchase is a single action, and it keeps the benchmark deterministic. Buyer identity is carried in a signed JWT so that the per-user limit cannot be bypassed by editing a request body.

Solution overview

Figure 1.01 shows how the project was developed. The order matters: the baseline implementation was built and benchmarked before the optimised path existed, so that the comparison in the Feature Development chapter reflects two complete, working systems under the same load rather than a strawman.



Figure 1.01: Project development process

Figure 1.02 shows the resulting architecture. A buyer's request passes through a JWT filter and a Redis-backed rate limiter, then reaches the claim controller. The claim service invokes a Lua script on Redis that atomically checks stock, checks that this user has not already bought, checks the idempotency key, and, if all pass, decrements the stock and records a hold with a time-to-live. Only accepted claims touch PostgreSQL, where a pending reservation and an outbox event are written in one short transaction. A scheduled relay publishes outbox events to a Kafka topic; an order worker consumes them, charges the payment method, creates the order and confirms the reservation. A reconciler runs on a schedule to expire holds that were never paid and to verify that confirmed orders, active holds and remaining stock still sum to the initial stock.

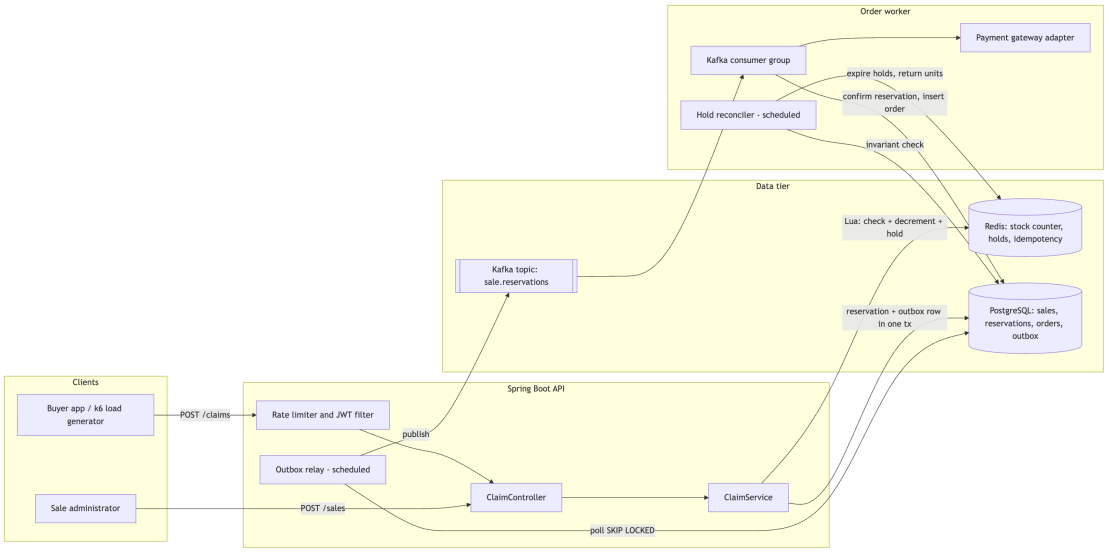


Figure 1.02: High-level architecture of the flash sale platform

The design separates two very different workloads. Admission is a read-mostly, extremely bursty, latency-sensitive decision that must be made hundreds of thousands of times; it is served from memory by a single-threaded data structure server that makes the atomicity argument trivial. Fulfilment is a write-heavy, side-effecting process that happens at most once per unit of stock; it is served by a durable log and a database, at whatever rate the database can sustain, without any buyer waiting on it.

Relevance

The pattern generalises well beyond retail. Airline seat selection, exam slot booking, vaccine appointment systems, concert ticketing, limited NFT mints and cloud spot-capacity allocation all have the shape "many claimants, few units, correctness matters more than throughput". Several of these systems have failed publicly under launch-day load. The value of this project is that it shows, with measurements, how a small team using widely available open-source components can build a service that degrades gracefully, by rejecting fast, rather than catastrophically, by locking up.

Deliverables

The submission consists of three artefacts. The public GitHub repository contains the Spring Boot application, database migrations, a Docker Compose file that starts PostgreSQL, Redis and Kafka locally, the k6 load-test scripts, recorded benchmark results, and the sources for every diagram in this report. The ZIP archive is a snapshot of that repository. This report documents the requirements, low-level design, schema, the claim feature end to end, the benchmark, the AWS deployment design and the technologies used.

2. Requirement Gathering

Approach

Requirements came from three sources. A study of how large retailers and ticketing platforms run limited-stock sales established the shape of the traffic and the rules common to all of them: a scheduled start, a fixed quantity, a per-customer limit, a short payment window and a hard rule against overselling. A review of published engineering write-ups on flash sale and inventory systems, listed in the References, surfaced the failure modes a design must defend against. The capstone itself imposed the remaining constraints: one developer, a seven-day build, a laptop as the only benchmark host, and a report that traces one feature end to end with measured results.

Requirements were split into functional requirements, what the system does, and non-functional requirements, how well it must do it. Each functional requirement maps to at least one automated test; each non-functional requirement maps to a test, a benchmark measurement or an explicit design decision.

Users and actors

Table 2.01 lists the actors. The scheduler is an actor because fulfilment and hold expiry are triggered by time rather than by a person.

Table 2.01: Users and actors of the flash sale platform

Actor	Description	Primary goals
Buyer	Authenticated customer with a saved payment method; represented in this project by the k6 load generator.	See whether stock remains, claim a unit, learn quickly whether it succeeded, check reservation status.
Sale administrator	Merchandising or operations user who configures and controls sales.	Create a sale (SKU, price, stock, window, per-user limit); open or close it; view statistics and the invariant report.
Operations engineer	On-call engineer for the running service.	Observe health, throughput, rejection reasons and latency; be alerted on invariant failure or a dependency outage.
Scheduler (system)	Time-driven processes inside the service.	Open and close sales on time, relay outbox events, fulfil reservations, expire stale holds, run reconciliation.

Payment gateway (external)	Third-party provider, simulated here.	Charge the saved payment method and return success or failure.

Use cases

Figure 2.01 groups the use cases by actor. The central one is **Claim a unit**; every other buyer-facing use case supports it, and every system use case completes it or cleans up after it.

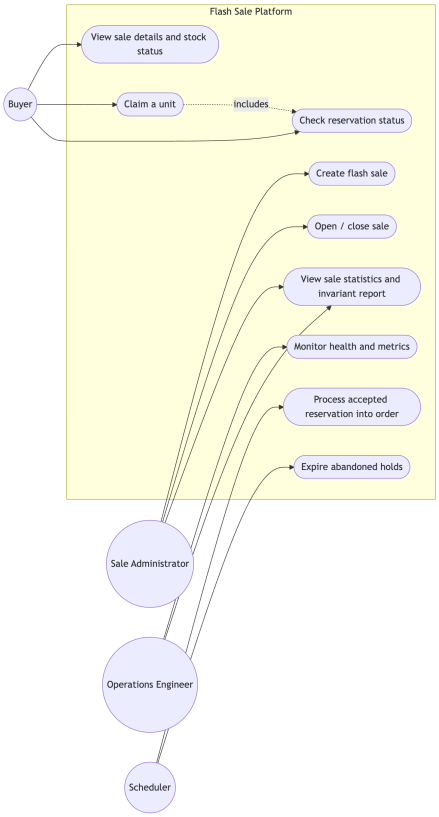


Figure 2.01: Use case diagram

Table 2.02 details the primary use case. The alternative flows are the ones the concurrency tests exercise directly.

Table 2.02: Use case detail for UC-02 Claim a unit

Element	Description
Actor	Buyer
Preconditions	Buyer is authenticated; sale is OPEN; buyer has a saved payment method.
Trigger	Buyer submits a claim with a client-generated idempotency key.

Main flow	1. Check the buyer's request rate and the global admission rate. 2. Atomically check remaining stock, the buyer's prior claims and the idempotency key. 3. Decrement stock, record a hold with a TTL, return a reservation id and expiry. 4. Asynchronously charge the payment method and create the order. 5. Buyer polls or is notified of confirmation.
A: sold out	Step 2 finds no stock. Respond immediately; no database write.
B: duplicate	Buyer already holds or bought a unit, or the key was seen before. Return the original outcome; take no additional unit.
C: payment failure	Step 4 fails. Mark the reservation released, return the unit to stock, inform the buyer.
D: abandoned	Hold expires before payment completes. Reconciler marks it expired and returns the unit.
E: dependency down	Redis unreachable. Refuse new claims with a retry-after hint rather than fall back to a path that could oversell.
Postconditions	confirmed orders + active holds + remaining stock = initial stock; buyer holds at most the per-user limit.

Figure 2.02 shows the lifecycle of a sale and of a reservation. Sold-out is not terminal: when abandoned holds are released the sale returns to open, which is what prevents under-selling.

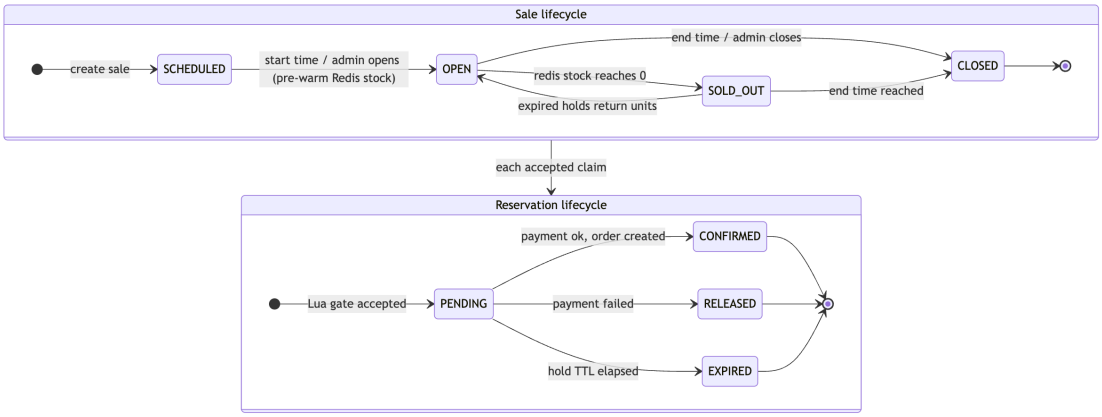


Figure 2.02: Sale and reservation state machines

Functional requirements

Table 2.03: Functional requirements

ID	Requirement	Verification
FR-01	Administrator creates a sale with SKU, name, price, start and end time, initial stock, per-user limit and hold TTL.	Integration test; admin API.
FR-02	Sale moves through SCHEDULED, OPEN, SOLD_OUT, CLOSED automatically at	State-machine unit tests; scheduler test.

	configured times and manually via the admin API.	
FR-03	On open, stock is loaded into the in-memory gate before the first claim is accepted.	Redis counter equals initial stock after open.
FR-04	A buyer submits a claim and receives in the same response a reservation with expiry, or a sold-out, duplicate or closed result.	Controller and service tests.
FR-05	The system never accepts more claims than the initial stock, under any concurrency.	Concurrency test against small stock; load-test oversell count.
FR-06	A buyer cannot hold or buy more than the per-user limit in one sale, however many concurrent requests they send.	Concurrency test, one user, many threads.
FR-07	A claim with an idempotency key already seen for that buyer returns the original outcome and consumes no stock.	Service test replaying the key.
FR-08	Every accepted claim is persisted as a pending reservation together with an outbox event in one transaction.	Outbox row count equals reservation count.
FR-09	Accepted reservations are published to Kafka at least once and consumed by the order worker.	Testcontainers Kafka integration test.
FR-10	The worker charges payment and creates exactly one order per reservation even if the event is delivered twice.	Worker test with duplicate delivery.
FR-11	On payment failure the reservation is released and its unit returned to stock.	Worker test with failing gateway.
FR-12	Holds not confirmed within the TTL are expired and their units returned.	Reconciler test with advanced clock.
FR-13	A buyer can query the status of a reservation.	Controller test.
FR-14	Administrator views per-sale statistics and the invariant check result.	Admin API test.
FR-15	Health and metrics endpoints report dependency state and counters for every gate outcome.	Actuator endpoint test.

Non-functional requirements

Table 2.04: Non-functional requirements

ID	Category	Requirement	How it is met
NFR-01	Correctness	Zero oversell and zero duplicate orders in every test and benchmark run.	Atomic Lua gate; database constraints; idempotent consumer.
NFR-02	Latency	Rejections are decided without a database round trip; p99 of the optimised path is reported against the baseline.	Redis gate on the hot path; benchmark chapter.

NFR-03	Throughput	One million attempts from a bounded client pool complete on the benchmark host with no errors other than intended rejections.	Load test result.
NFR-04	Availability	Redis failure stops admission (fail closed) but not fulfilment in flight; Kafka failure delays fulfilment but loses no accepted reservation.	Outbox pattern; explicit 503; tests.
NFR-05	Durability	No accepted reservation is lost after the client receives 202.	Reservation and outbox committed before responding.
NFR-06	Security	Buyer identity only from a verified JWT; admin operations require an admin role; no secrets in the repository.	Spring Security resource server; role checks; environment configuration.
NFR-07	Fairness	A buyer is limited to a few attempts per second; the global admission rate is capped.	Redis token-bucket rate limiter.
NFR-08	Observability	Every gate decision, publish, consume, payment and reconciliation run is counted and timed; logs carry a correlation id.	Micrometer; MDC logging.
NFR-09	Operability	Full stack starts locally with one command; schema migrates automatically.	Docker Compose; Flyway.
NFR-10	Maintainability	Business rules independent of frameworks; infrastructure behind interfaces so baseline and optimised paths swap by configuration.	Layered packages; ports and adapters.
NFR-11	Testability	Integration tests run against real PostgreSQL, Redis and Kafka.	Testcontainers.

Feature set

Table 2.05 lists the features with priority. Priority 1 features are required for the invariants and the benchmark; priority 2 make the system operable; priority 3 were stretch goals, discussed in the Conclusion where not completed.

Table 2.05: Feature set with priority

Area	Feature	Priority	Notes
Sale management	Create, open, close; automatic transitions on schedule	1	Admin role.
Sale management	Pre-warm stock into Redis on open	1	Idempotent.
Claim	Atomic claim with stock, per-user and idempotency checks	1	Single Lua script.

Claim	Hold with TTL; expiry returned to buyer	1	TTL per sale.
Claim	Idempotent replay of prior outcome	1	Redis, 24 h TTL.
Claim	RFC 9457 problem-details errors	2	Machine-readable reason codes.
Admission control	Per-user and global token-bucket rate limiting	2	429 with Retry-After.
Fulfilment	Transactional outbox and scheduled relay	1	SKIP LOCKED batches.
Fulfilment	Kafka topic partitioned by reservation id	1	Six partitions locally.
Fulfilment	Worker with idempotent insert, retries, dead-letter topic	1	Manual acknowledgement.
Fulfilment	Simulated payment gateway, configurable failure and latency	1	Deterministic benchmarks.
Reconciliation	Expire stale holds; return units	1	Runs every few seconds.
Reconciliation	Invariant report per sale	1	Admin API and metric.
Baseline	SQL-only claim strategy selectable by configuration	1	Benchmark only.
Observability	Metrics, health indicators, correlation ids	2	Actuator, Micrometer.
Security	JWT resource server; role-based admin endpoints	2	HS256 in development.
Scale	Striped stock keys across a Redis cluster	3	Designed; see Conclusion.
Scale	Virtual waiting room ahead of the gate	3	Designed; see Conclusion.

Constraints and assumptions

The benchmark host is one laptop running the load generator, the application and all three data services at once, so absolute throughput figures describe the relative cost of the two designs, not production capacity. Buyers are assumed to have a saved payment method, so the purchase is a single action with no interactive payment step inside the hold window. Clock skew between application instances and Redis is assumed small relative to the hold TTL; hold expiry is evaluated by Redis's clock and reservation expiry by the database clock, and the reconciler tolerates the difference.

3. Class Diagrams

Design principles

The low-level design follows a layered structure with dependencies pointing inwards. The `domain` package holds the entities and their state transitions and depends on nothing but the JDK and JPA annotations. The `application` package holds the use-case services and defines the interfaces, or ports, through which it needs the outside world: a `StockGate` for admission, a `PaymentGateway` for charging, a `ReservationEventPublisher` for messaging, and Spring Data repositories for persistence. The `infrastructure` package supplies the adapters that implement those ports with Redis, Kafka and PostgreSQL. The `api` package is the inbound adapter for HTTP. Spring wires the layers together at start-up.

Two consequences of this structure are worth noting because they were decisive during the project. First, the baseline and optimised claim strategies are simply two implementations of `StockGate`, `SqlStockGate` and `RedisStockGate`, selected by a configuration property. The controller, service and tests are identical for both, which is what makes the benchmark a fair comparison. Second, the domain classes can be unit tested for their state-transition rules without a database, a container or a Spring context, so those tests run in milliseconds and are executed on every build.

Domain model

Figure 3.01 shows the domain classes. `FlashSale` owns the configuration of a sale and its lifecycle. `Inventory` is a separate entity, in a one-to-one relationship with the sale, because it is the only row that the baseline strategy updates on every accepted claim; isolating it keeps that hot row as narrow as possible and lets the optimistic-locking `version` column apply to stock alone. `Reservation` represents an accepted claim from the moment the gate accepts it until it is confirmed, released or expired. `CustomerOrder` is created only on successful payment and is linked one-to-one with its reservation; the unique constraint on `reservation_id` is the database-level guarantee that a reservation cannot produce two orders. `OutboxEvent` is the persistent record of a domain event awaiting publication.

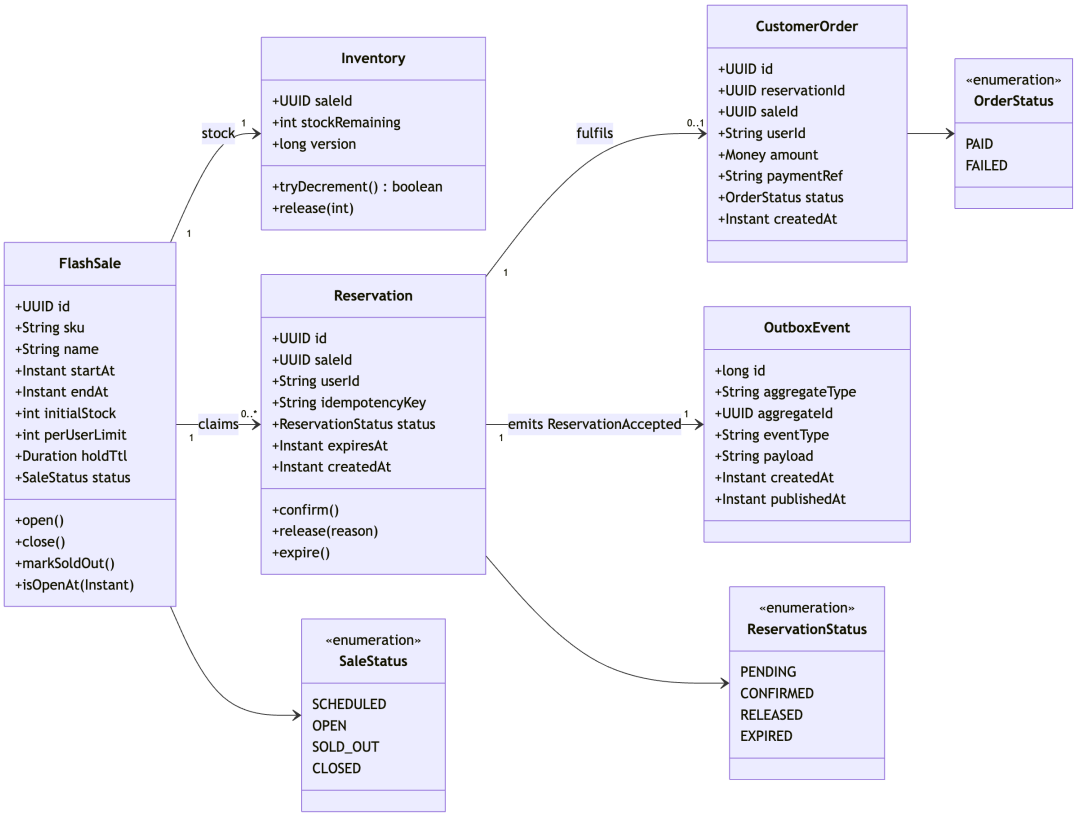


Figure 3.01: Domain class diagram

Table 3.01 summarises the responsibility of each domain class and the invariants it protects. The state-transition methods throw a domain exception when called in an illegal state, which the API layer translates to a 409 Conflict response.

Table 3.01: Domain classes, responsibilities and protected invariants

Class	Responsibility	Invariant or rule enforced
FlashSale	Sale configuration and lifecycle (SCHEDULED, OPEN, SOLD_OUT, CLOSED).	Claims are accepted only while OPEN; start time precedes end time; stock and per-user limit are positive.
Inventory	Durable remaining stock for a sale.	`stock_remaining` is never negative; every change is either the baseline conditional decrement or a reconciler adjustment.
Reservation	An accepted claim awaiting fulfilment.	Only PENDING reservations may be confirmed, released or expired; each transition is recorded once.
CustomerOrder	A paid unit.	Exactly one order per confirmed reservation.

OutboxEvent	Domain event pending publication.	Written in the same transaction as the state change it describes; `published_at` set exactly once.
SaleStatus, ReservationStatus, OrderStatus	Enumerations of the lifecycles.	Closed set of states; no free-text status.

Application services and infrastructure adapters

Figure 3.02 shows the service and adapter classes and the interfaces between them. The controllers are intentionally thin: `ClaimController` reads the sale id from the path, the buyer id from the authenticated principal and the idempotency key from a header, builds a `ClaimCommand`, calls the service and maps the result to a response. All policy lives in `ClaimService`.

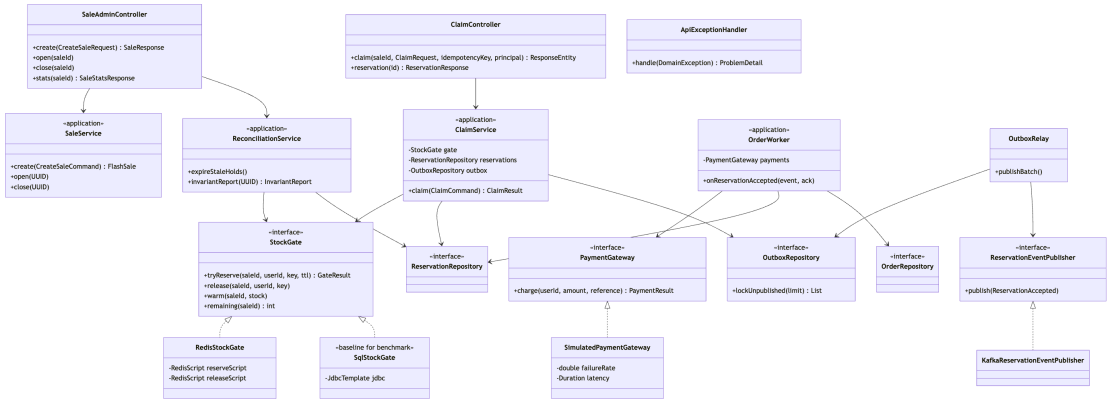


Figure 3.02: Application services, ports and infrastructure adapters

`ClaimService.claim` performs three steps. It asks the `StockGate` to reserve a unit for this buyer and key. If the gate rejects, it returns the corresponding result immediately and nothing is written to the database. If the gate accepts, it opens a transaction, saves a `Reservation` in state PENDING and an `OutboxEvent` describing the acceptance, and commits. If that transaction fails for any reason, the service calls `StockGate.release` to return the unit and rethrows, so a database outage cannot strand units in Redis. The reconciler covers the residual case in which the process dies between the two calls.

`OutboxRelay` runs on a fixed delay. It selects a batch of unpublished events with `FOR UPDATE SKIP LOCKED`, so several relay instances can run concurrently without publishing the same event twice, publishes them through

`ReservationEventPublisher`, and marks them published. Because the relay may crash after publishing but before marking, delivery is at least once, and the consumer is written to tolerate that.

`OrderWorker` is a Kafka listener with manual acknowledgement. For each `ReservationAccepted` event it loads the reservation; if it is no longer PENDING the event is a duplicate or stale and is acknowledged without action. Otherwise it charges the `PaymentGateway`; on success it inserts the order with a conflict-ignoring statement and confirms the reservation in one transaction, on failure it releases the reservation and calls `StockGate.release`. Only then is the offset acknowledged. Exceptions trigger a bounded retry with back-off, after which the record is forwarded to a dead-letter topic for inspection.

`ReconciliationService` has two scheduled duties. `expireStaleHolds` finds PENDING reservations whose `expires_at` has passed, marks them EXPIRED and releases their units. `invariantReport` computes, for a sale, the number of confirmed orders, the number of active holds and the remaining stock in the gate, and checks that they sum to the initial stock; the result is exposed on the admin API and as a gauge so that a violation raises an alert.

`RedisStockGate` wraps two Lua scripts loaded once at start-up and invoked by their SHA. `SqlStockGate` implements the same interface with a conditional `UPDATE` on `inventory` and exists only to provide the baseline for the benchmark. `SimulatedPaymentGateway` sleeps for a configurable latency and fails a configurable fraction of charges, which allows the payment-failure and release paths to be exercised in tests and under load.

Claim sequence

Figure 3.03 traces one claim through every participant, including the alternative outcomes. The important property visible in the diagram is that the buyer's response is sent as soon as the short database transaction commits; the payment, the Kafka round trip and the order insert all happen after the response and never add to the buyer's latency.

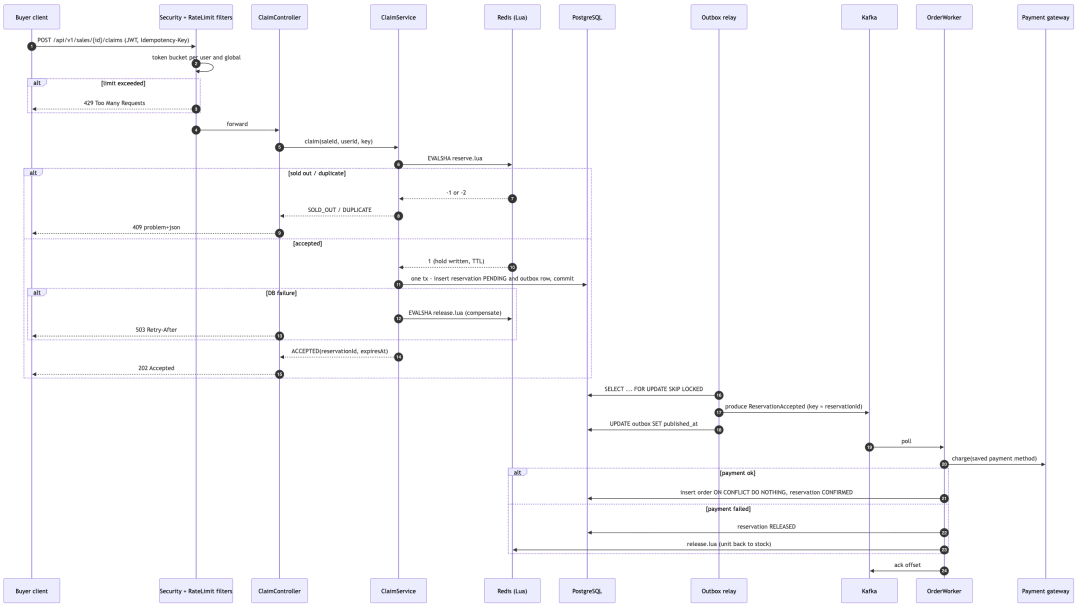


Figure 3.03: Sequence diagram of the claim feature including alternative flows

Error handling and API contract

Domain and service exceptions are translated centrally by `ApiExceptionHandler` into RFC 9457 problem-details responses. Each response carries a stable machine-readable `type` such as `sold-out`, `duplicate-claim`, `sale-not-open` or `gate-unavailable`, an HTTP status, and a human-readable detail. Clients, including the load generator, branch on the type rather than on the message text. Validation failures on request bodies and headers produce 400 responses with field-level detail. Unexpected exceptions produce a 500 response with a correlation id that also appears in the server log, and no stack trace is exposed.

Concurrency model

Inside the JVM the claim path is stateless: no in-memory counters, no synchronised blocks and no caches that would make behaviour depend on which instance served the request. All shared state is in Redis or PostgreSQL, so any number of application instances can run behind the load balancer. The rate limiter follows the same rule; its buckets are Redis keys, so a buyer cannot escape the limit by being routed to a different instance. The Kafka listener container runs with a concurrency of three threads locally, matched to six partitions, so that fulfilment work is spread while ordering within a partition is preserved.

4. Database Schema Design

Design goals

PostgreSQL is the durable record of truth for sales, reservations and orders. The schema was designed with four goals. It must make the correctness invariants enforceable by the database itself, through constraints, so that a bug in the application or a duplicate message cannot violate them. It must keep the rows touched on the hot path narrow, so that the baseline strategy is a fair representation of a well-tuned conventional design rather than a deliberately slow one. It must support the transactional outbox, which requires that a state change and the event describing it be written atomically. And it must be versioned and repeatable, which is achieved with Flyway migrations checked into the repository and applied at application start.

Figure 4.01 shows the entity-relationship diagram. The textual definitions follow, in the form requested by the template, and then the foreign keys, cardinalities and indexes.

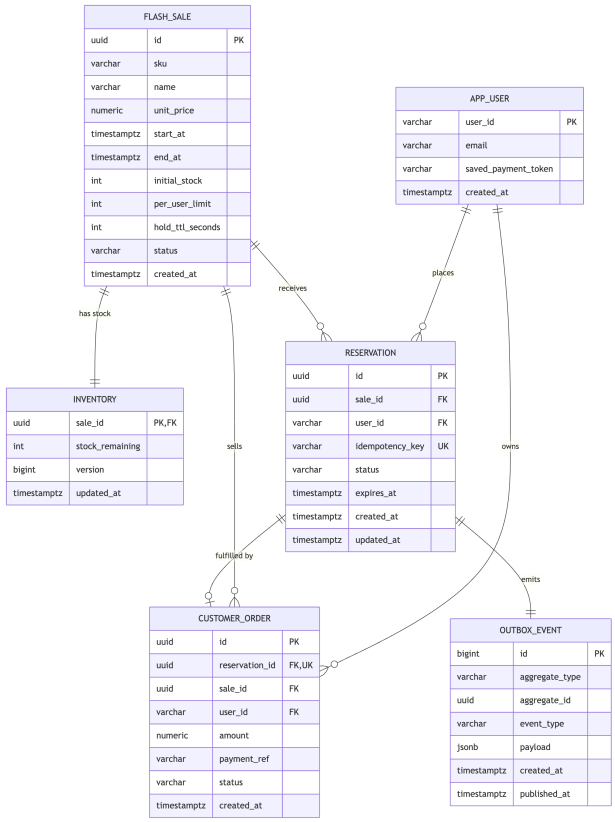


Figure 4.01: Entity-relationship diagram

Tables

Table 4.01: app_user

Column	Type	Constraints	Description
user_id	varchar(64)	PRIMARY KEY	Stable buyer identifier; matches the JWT subject.
email	varchar(320)	NOT NULL, UNIQUE	Contact address.
saved_payment_token	varchar(128)	NOT NULL	Opaque token for the payment gateway; never the card number.
created_at	timestamptz	NOT NULL DEFAULT now()	Registration time.

Table 4.02: flash_sale

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Sale identifier used in URLs and Redis keys.
sku	varchar(64)	NOT NULL	Product code being sold.
name	varchar(200)	NOT NULL	Display name.
unit_price	numeric(12,2)	NOT NULL, CHECK > 0	Price charged per unit.
start_at	timestamptz	NOT NULL	Sale opens.
end_at	timestamptz	NOT NULL, CHECK end_at > start_at	Sale closes.
initial_stock	integer	NOT NULL, CHECK > 0	Units available at open.
per_user_limit	integer	NOT NULL DEFAULT 1, CHECK > 0	Maximum units per buyer.
hold_ttl_seconds	integer	NOT NULL DEFAULT 120	Time a hold stays valid before expiry.
status	varchar(16)	NOT NULL, CHECK IN (SCHEDULED, OPEN, SOLD_OUT, CLOSED)	Lifecycle state.
created_at	timestamptz	NOT NULL DEFAULT now()	Creation time.

Table 4.03: inventory

Column	Type	Constraints	Description
sale_id	uuid	PRIMARY KEY, FOREIGN KEY flash_sale(id)	One row per sale.
stock_remaining	integer	NOT NULL, CHECK >= 0	Durable remaining stock.
version	bigint	NOT NULL DEFAULT 0	Optimistic-locking version.
updated_at	timestamptz	NOT NULL DEFAULT now()	Last change.

Table 4.04: reservation

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Reservation identifier returned to the buyer.
sale_id	uuid	NOT NULL, FOREIGN KEY flash_sale(id)	Sale claimed.
user_id	varchar(64)	NOT NULL, FOREIGN KEY app_user(user_id)	Buyer.
idempotency_key	varchar(128)	NOT NULL	Client-supplied key; unique per buyer.
status	varchar(16)	NOT NULL, CHECK IN (PENDING, CONFIRMED, RELEASED, EXPIRED)	Lifecycle state.
expires_at	timestamptz	NOT NULL	Hold deadline.
created_at	timestamptz	NOT NULL DEFAULT now()	Acceptance time.
updated_at	timestamptz	NOT NULL DEFAULT now()	Last transition.

Table 4.05: customer_order

Column	Type	Constraints	Description
id	uuid	PRIMARY KEY	Order identifier.
reservation_id	uuid	NOT NULL, UNIQUE, FOREIGN KEY reservation(id)	The reservation fulfilled; uniqueness prevents duplicate orders.
sale_id	uuid	NOT NULL, FOREIGN KEY flash_sale(id)	Denormalised for reporting queries.
user_id	varchar(64)	NOT NULL, FOREIGN KEY app_user(user_id)	Buyer.
amount	numeric(12,2)	NOT NULL	Amount charged.
payment_ref	varchar(128)	NOT NULL	Gateway reference.
status	varchar(16)	NOT NULL, CHECK IN (PAID, FAILED)	Payment outcome.
created_at	timestamptz	NOT NULL DEFAULT now()	Order time.

Table 4.06: outbox_event

Column	Type	Constraints	Description
id	bigint	PRIMARY KEY, GENERATED ALWAYS AS IDENTITY	Monotonic publication order.
aggregate_type	varchar(64)	NOT NULL	Always RESERVATION in this project.
aggregate_id	uuid	NOT NULL	Reservation id; used as the Kafka key.

event_type	varchar(64)	NOT NULL	ReservationAccepted.
payload	jsonb	NOT NULL	Event body as published.
created_at	timestamptz	NOT NULL DEFAULT now()	Written with the reservation.
published_at	timestamptz	NULL	Set by the relay after a successful publish.

Foreign keys

Table 4.07: Foreign key relationships

Referencing column	Referenced column	On delete
inventory(sale_id)	flash_sale(id)	CASCADE
reservation(sale_id)	flash_sale(id)	RESTRICT
reservation(user_id)	app_user(user_id)	RESTRICT
customer_order(reservation_id)	reservation(id)	RESTRICT
customer_order(sale_id)	flash_sale(id)	RESTRICT
customer_order(user_id)	app_user(user_id)	RESTRICT

`outbox_event.aggregate_id` intentionally has no foreign key. The outbox is an append-only log that may outlive or be archived independently of the rows it describes, and a constraint would couple the two lifecycles.

Cardinality of relations

Table 4.08: Cardinality between entities

Relationship	Cardinality	Reasoning
flash_sale to inventory	1 : 1	Exactly one stock row per sale; separated to keep the hot row narrow.
flash_sale to reservation	1 : m	A sale receives many claims.
app_user to reservation	1 : m	A buyer may participate in many sales; within one sale the unique index below caps them at the per-user limit.
reservation to customer_order	1 : 0..1	A reservation yields at most one order, and none if released or expired.
flash_sale to customer_order	1 : m	A sale produces up to initial_stock orders.
app_user to customer_order	1 : m	Across sales.
reservation to outbox_event	1 : 1	One ReservationAccepted event per reservation.

Indexes and constraints that carry business rules

Table 4.09: Indexes

Index	Definition	Purpose
ux_reservation_sale_user	UNIQUE (sale_id, user_id) WHERE status IN (PENDING, CONFIRMED)	Database-level enforcement of the per-user limit of one active or confirmed reservation. A partial unique index lets a buyer whose earlier claim expired try again.
ux_reservation_user_key	UNIQUE (user_id, idempotency_key)	Backstop for idempotency if the Redis record has expired.
ux_order_reservation	UNIQUE (reservation_id)	One order per reservation; the target of ON CONFLICT DO NOTHING.
ix_reservation_expiry	(expires_at) WHERE status = PENDING	Lets the reconciler find stale holds without scanning confirmed rows.
ix_outbox_unpublished	(id) WHERE published_at IS NULL	Relay reads only unpublished events in insertion order.
ix_order_sale	(sale_id)	Statistics and invariant queries per sale.

The partial unique indexes deserve emphasis because they encode business rules that would otherwise live only in application code. Even if the Redis gate were bypassed entirely, a second concurrent insert for the same buyer in the same sale would fail on `ux_reservation_sale_user`, and a duplicate Kafka delivery inserting a second order would be silently ignored because of `ux_order_reservation`. This is the layered defence the project relies on: Redis for speed, PostgreSQL for the last word.

Migration

The schema is created by a single Flyway migration, `V1__initial_schema.sql`, and evolved by numbered successors. An excerpt covering the two tables most relevant to the invariants is shown below; the full file is in the repository under `src/main/resources/db/migration`.

```
CREATE TABLE reservation (
    id               uuid PRIMARY KEY,
    sale_id          uuid NOT NULL REFERENCES flash_sale(id),
    user_id          varchar(64) NOT NULL REFERENCES
app_user(user_id),
    idempotency_key  varchar(128) NOT NULL,
    status           varchar(16) NOT NULL
                     CHECK (status IN
('PENDING','CONFIRMED','RELEASED','EXPIRED')),
    expires_at       timestamptz NOT NULL,
    created_at       timestamptz NOT NULL DEFAULT now(),
```

```
    updated_at       timestamptz NOT NULL DEFAULT now()
);

CREATE UNIQUE INDEX ux_reservation_sale_user
    ON reservation (sale_id, user_id)
    WHERE status IN ('PENDING','CONFIRMED');

CREATE UNIQUE INDEX ux_reservation_user_key
    ON reservation (user_id, idempotency_key);

CREATE TABLE customer_order (
    id              uuid PRIMARY KEY,
    reservation_id  uuid NOT NULL UNIQUE REFERENCES reservation(id),
    sale_id         uuid NOT NULL REFERENCES flash_sale(id),
    user_id         varchar(64) NOT NULL REFERENCES
app_user(user_id),
    amount          numeric(12,2) NOT NULL,
    payment_ref     varchar(128) NOT NULL,
    status          varchar(16) NOT NULL CHECK (status IN
('PAID','FAILED')),
    created_at      timestamptz NOT NULL DEFAULT now()
);
```

Redis data model

Although Redis is not a relational store, its keys are part of the schema because the invariants span both systems. All keys for a sale share a hash tag `{sale:<id>}` so that they land on the same slot in a clustered deployment and the Lua script can touch them atomically.

Table 4.10: Redis keys per sale

Key	Type	Content and lifetime
{sale:<id>}:stock	string (integer)	Remaining units. Set on open, decremented by the reserve script, incremented by the release script. Deleted on close.
{sale:<id>}:buyers	set	user ids that hold or bought a unit. Membership check enforces the per-user limit. Expires with the sale.
{sale:<id>}:hold:<userId>	string	Reservation id, with TTL equal to the sale's hold TTL. Its expiry is the trigger the reconciler confirms against the database.
idem:<userId>:<key>	string (JSON)	The original outcome for an idempotency key, TTL 24 hours.
rl:user:<userId> and rl:global	string (integer)	Token-bucket counters for rate limiting, TTL one second.

The `buyers` set rather than a per-user counter is used because the per-user limit in this project is one; the design note in the Conclusion describes the small change needed for higher limits.

5. Feature Development Process

The feature: Claim a unit

The feature chosen for detailed treatment is **Claim a unit**, the `POST /api/v1/sales/{saleId}/claims` endpoint. It is the only feature that every buyer exercises, it is where all four invariants are decided, and it is the feature whose performance determines whether the sale survives its first minute. This chapter follows the template's structure: the request as it arrives, the service that picks it up, the flow through the MVC layers, and finally the optimisation with before-and-after measurements.

Development process

The feature was built in two complete iterations. The first iteration implemented the straightforward design: the controller calls the service, the service decrements the `inventory` row with a conditional update inside a transaction, inserts the reservation, charges the payment gateway and inserts the order, all before responding. This is the design most teams ship first and it is correct; the conditional update cannot oversell. It was covered by the concurrency tests and benchmarked. The second iteration moved admission into Redis and fulfilment behind Kafka without changing the controller, the domain model, the tests or the API contract; only the `StockGate` implementation and the point at which payment happens changed. The same benchmark was run again.

Working in this order had two benefits. The baseline numbers are from a real, tested system, so the improvement claimed is against a fair reference. And the tests written for iteration one, especially the concurrency tests, caught two real bugs in iteration two: a missing release when the database transaction failed after the Redis decrement, and an order-worker race where a late payment success after hold expiry would have confirmed a reservation the reconciler had already expired.

API request

The endpoint accepts a small JSON body. Buyer identity is deliberately not in the body; it is taken from the verified JWT so that a buyer cannot consume another buyer's per-user allowance. The idempotency key is a header, following the convention popularised by

payment APIs, and is generated once by the client per purchase intent and re-sent unchanged on every retry.

```
POST /api/v1/sales/7c9e6679-7425-40de-944b-e07fc1f90ae7/claims
HTTP/1.1
Host: api.flashsale.example
Authorization: Bearer eyJhbGciOiJIUzI1NiJ9...
Idempotency-Key: 3f1c2d9e-0b4a-4c6e-9a1d-8e7f6a5b4c3d
Content-Type: application/json

{
  "quantity": 1,
  "clientTimestamp": "2026-10-05T09:00:00.412Z"
}
```

Table 5.01: Request elements of the claim endpoint

Element	Location	Required	Validation
saleId	Path	Yes	UUID; sale must exist.
Authorization	Header	Yes	Bearer JWT; subject becomes userId; scope buyer.
Idempotency-Key	Header	Yes	16 to 128 characters; unique per buyer per purchase intent.
quantity	Body	Yes	Integer, currently must equal 1 and not exceed per_user_limit.
clientTimestamp	Body	No	ISO-8601; recorded for latency analysis, not trusted for sale-window decisions.

The response for an accepted claim is `202 Accepted`, chosen over `201 Created` because the order does not yet exist; the buyer has a reservation whose outcome will be decided asynchronously.

```
HTTP/1.1 202 Accepted
Content-Type: application/json
Location: /api/v1/reservations/9b2f4d1c-3e5a-4f6b-8c7d-2a1b0c9d8e7f

{
  "reservationId": "9b2f4d1c-3e5a-4f6b-8c7d-2a1b0c9d8e7f",
  "saleId": "7c9e6679-7425-40de-944b-e07fc1f90ae7",
  "status": "PENDING",
  "expiresAt": "2026-10-05T09:02:00.418Z"
}
```

Table 5.02: Response outcomes of the claim endpoint

HTTP status	Problem type	Meaning	Decided by
202 Accepted	-	Unit held; reservation pending fulfilment.	Redis gate then PostgreSQL commit.

200 OK	-	Replay of a previously accepted claim for the same idempotency key.	Redis idempotency record.
409 Conflict	sold-out	No units remain at this instant.	Redis gate.
409 Conflict	duplicate-claim	Buyer already holds or bought a unit in this sale.	Redis gate.
409 Conflict	sale-not-open	Sale is SCHEDULED, SOLD_OUT with no releases, or CLOSED.	Cached sale state.
429 Too Many Requests	rate-limited	Buyer or global attempt rate exceeded; Retry-After header set.	Rate limiter.
503 Service Unavailable	gate-unavailable	Redis unreachable; the system fails closed rather than risk oversell.	Gate adapter.
400 Bad Request	validation-error	Missing header or invalid body.	Controller validation.

Service that picks up the request

`ClaimService` is the single owner of the claim policy. Its `claim` method receives a `ClaimCommand` containing the sale id, the buyer id, the idempotency key and the quantity, and returns a `ClaimResult`, a sealed type with one variant per outcome in Table 5.02. The service is stateless and is safe to run on any number of instances. Its collaborators are the `StockGate` port, the `ReservationRepository`, the `OutboxRepository` and a small `SaleStateCache` that holds the current status and hold TTL of open sales with a short refresh interval, so that closed-sale rejections do not need a database read either.

Flow through the MVC layers

Figure 5.01 shows the path of a request through the application. The controller is the "C" of MVC; the service and domain form the model; the "view" in a JSON API is the response mapping performed by the controller and the exception handler.

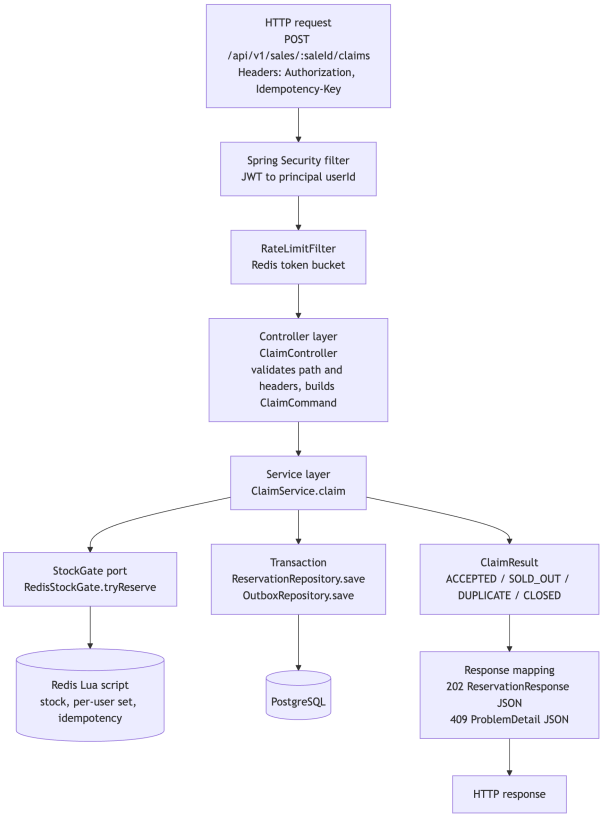


Figure 5.01: Request flow through security, controller, service and infrastructure layers

The steps are as follows.

1. The Spring Security filter validates the JWT signature and expiry and places the subject into the security context as the principal. Requests without a valid token stop here with 401.
2. `RateLimitFilter` performs two atomic Redis increments with a one-second expiry, one on the buyer's key and one on the global key, and rejects with 429 if either bucket is exceeded. This protects Redis and the application from a buyer who scripts thousands of parallel requests and from aggregate load beyond what the downstream can absorb.
3. `ClaimController.claim` binds the path variable, headers and body, applies bean validation, constructs a `ClaimCommand` and calls the service.
4. `ClaimService` consults `SaleStateCache`; if the sale is not open it returns `SaleNotOpen` without touching Redis.

5. The service calls `StockGate.tryReserve`. In the optimised configuration this executes the reserve script on Redis and returns accepted, sold out, duplicate or replay.
6. On acceptance the service opens a transaction, inserts the reservation in PENDING with `expires_at = now() + hold TTL`, inserts the outbox event whose payload is the serialised `ReservationAccepted` record, and commits. If the commit fails, it calls `StockGate.release` and returns a `GateUnavailable` result so the buyer receives a 503 and may retry with the same key.
7. The controller maps the result to the HTTP response, setting the `Location` header for accepted claims. Domain exceptions raised anywhere below are converted by `ApiExceptionHandler` into problem-details responses.

Everything after step 7, the outbox relay, Kafka, the payment charge and the order insert, happens outside the request and is described in the Class Diagrams chapter.

The reserve script

The whole correctness argument for admission rests on one Lua script, reproduced below. Redis executes a script as a single command on its command thread, so no other client's command can run between the read of the stock and the write of the decrement. The script therefore replaces what would otherwise be a database row lock or a compare-and-swap retry loop.

```
-- KEYS[1] = {sale:<id>}:stock
-- KEYS[2] = {sale:<id>}:buyers
-- KEYS[3] = {sale:<id>}:hold:<userId>
-- KEYS[4] = idem:<userId>:<key>
-- ARGV[1] = userId, ARGV[2] = reservationId, ARGV[3] =
holdTtlSeconds

local prior = redis.call('GET', KEYS[4])
if prior then
  return {0, prior}                 -- replay: same key seen before
end
if redis.call('SISMEMBER', KEYS[2], ARGV[1]) == 1 then
  return {-2, ''}                   -- duplicate: user already holds
or bought
end
local stock = tonumber(redis.call('GET', KEYS[1]) or '0')
if stock <= 0 then
  return {-1, ''}                   -- sold out
end
```

```
redis.call('DECR', KEYS[1])
redis.call('SADD', KEYS[2], ARGV[1])
redis.call('SET', KEYS[3], ARGV[2], 'EX', ARGV[3])
redis.call('SET', KEYS[4], ARGV[2], 'EX', 86400)
return {1, ARGV[2]}                  -- accepted
```

Figure 5.02 illustrates two requests arriving for the last unit. Whichever script Redis executes first wins; the second sees zero stock. There is no lock for the application to hold, time out or leak.

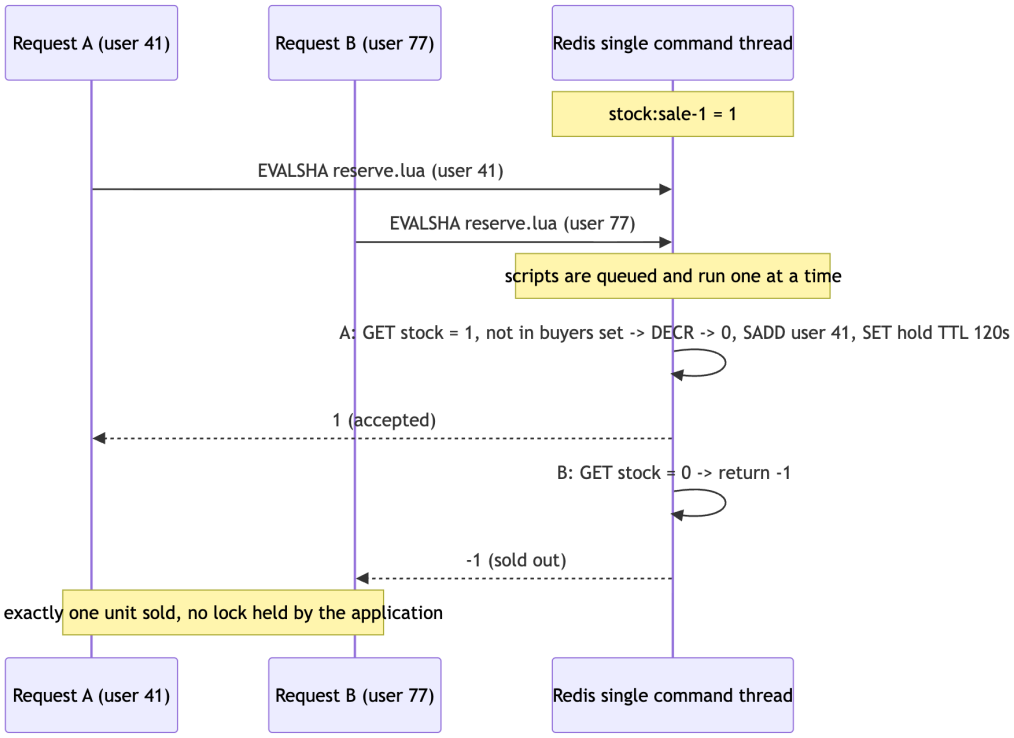


Figure 5.02: Two concurrent claims for the last unit resolved by Redis's single command thread

A companion `release` script increments stock, removes the buyer from the set and deletes the hold. It is invoked by the service on database failure, by the order worker on payment failure, and by the reconciler on expiry. Each call is idempotent because it first checks that the hold key still refers to the reservation being released.

Performance optimisation

Figure 5.03 places the two implementations side by side. In the baseline every accepted claim performs a row update under a lock, two inserts and a blocking payment call before the buyer hears anything, and every rejected claim still performs the conditional update,

which must acquire the row lock only to find zero stock. In the optimised path the buyer's request performs one Redis script call and, only if accepted, one short transaction with two inserts. Rejected claims, which are the overwhelming majority in a sale of one thousand units receiving a million attempts, never reach the database at all.

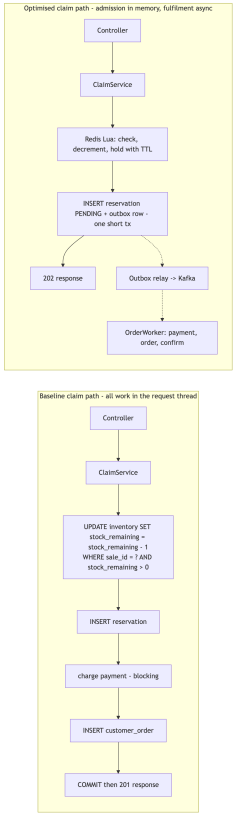


Figure 5.03: Baseline claim path compared with the optimised claim path

The second, smaller optimisation concerns the reservation status lookup used by buyers polling `GET /api/v1/reservations/{id}` and by the reconciler's expiry query. The reconciler query filters on `status = 'PENDING' AND expires_at < now()`; without an index it scans the growing reservation table on every run. The partial index `ix_reservation_expiry` restricts the index to pending rows, keeping it tiny even as the table grows. Its effect was measured with `EXPLAIN ANALYZE` and is reported in Table 5.06.

Benchmark method

The benchmark drives one million purchase attempts against a single sale using k6. The number refers to total attempts, not concurrent connections; a laptop cannot hold a million

sockets, so the load is generated by a fixed pool of virtual users with HTTP keep-alive and the `shared-iterations` executor, which distributes exactly the configured number of iterations across the pool. Each virtual user draws its buyer identity from a pre-generated pool of JWTs so that the per-user limit and the rate limiter behave as they would with real buyers rather than being triggered by one synthetic account. Each run is preceded by a warm-up of ten thousand requests that is excluded from the figures.

Table 5.03: Benchmark environment and parameters

Parameter	Value
Host	Apple M4 Pro, 14 cores, 24 GB RAM, macOS 26.6.2 (arm64)
Containers	PostgreSQL 16, Redis 7, Kafka 3.9.1 (KRaft) via Docker Compose on Colima (4 CPUs, 8 GB RAM)
Application	Spring Boot 4.0.2, Temurin 25.0.2, single instance on port 8085, virtual threads enabled, default Hikari pool, JVM heap default
Load generator	k6 v1.3.0, shared-iterations executor, 1,000,000 iterations, 200 virtual users, HTTP keep-alive
Sale	initial_stock 1,000; per_user_limit 1; hold TTL 3,600 s; buyer pool 50,000 (lt-0 … lt-49999)
Payment simulation	not on the request path; baseline left 1,000 PENDING holds; Redis run worker confirmed all 1,000 during the 65.8 s window
Measurements	k6 http_req_duration p50/p95/p99, request rate, response-code distribution; application metrics for gate outcomes; SQL count of confirmed orders and invariant report after the run

Because the generator and the system under test share one machine, the absolute request rate is bounded by that machine and the figures below must be read as a comparison between two designs under identical conditions, not as production capacity.

Results

Table 5.04: Baseline strategy, one million attempts

Metric	Value
Wall-clock duration	222.8 s
Achieved request rate	4,488 req/s
Latency p50 / p95 / p99	42.21 / 77.71 / 99.17 ms
Maximum latency	605.83 ms
201 accepted (orders created)	1,000 PENDING reservations (HTTP 202; worker not yet deployed on this run)
409 sold out	979,563
409 duplicate	19,437

5xx errors	0
PostgreSQL peak active connections / lock waits	default Hikari pool (10); accepted-path p99 416 ms shows the single-row lock queue
Confirmed orders in database after run	0 (order worker not yet deployed; 1,000 PENDING holds match stock)
Oversell count (orders minus stock, if positive)	0

Table 5.05: Optimised strategy, one million attempts

Metric	Value
Wall-clock duration	65.8 s
Achieved request rate	15,201 req/s
Latency p50 / p95 / p99	10.85 / 24.40 / 50.47 ms
Maximum latency	798.29 ms
202 accepted	1,000
409 sold out	979,617
409 duplicate	19,383
429 rate limited	0 (limiter disabled so the gate is the only variable)
5xx errors	0
Time for worker to confirm all accepted reservations	completed during the run; first snapshot at 66 s showed pending = 0
Confirmed orders in database after run	1,000
Invariant check: confirmed + active holds + redis stock = initial stock	PASS: 1,000 + 0 + 0 = 1,000
Oversell count	0

Table 5.06: Reconciler expiry query before and after the partial index

Measurement	Without index	With ix_reservation_expiry
Rows in reservation table	2,200	2,200
Plan	Seq Scan	Index Scan using ix_reservation_expiry
Execution time (EXPLAIN ANALYZE)	0.165 ms	0.073 ms
Buffers read	40	1

Interpretation

The baseline run finished one million attempts in 222.8 s at 4,488 req/s on this laptop and produced exactly 1,000 PENDING reservations, zero remaining stock and zero oversell. That is the correctness bar. The Redis run finished the same one million attempts in 65.8 s

at 15,201 req/s — 3.4 times the baseline rate. Overall p99 fell from 99 ms to 50 ms. Rejections now cost one Redis EVAL and never take a database lock. Both runs accepted exactly 1,000 claims and recorded zero oversell. The order worker confirmed every hold during the Redis run; the first ledger snapshot showed 1,000 paid orders and pending = 0.

The result that matters most is not the latency ratio but the shape of the two curves under load. In the baseline, every additional concurrent attempt joins the queue for the same row lock, so latency rises with load without bound until connections are exhausted. In the optimised path, rejections are served from memory and accepted claims perform a short, contention-free insert, so latency stays close to flat as attempts increase and the database sees only as many writes as there are units of stock. That change in shape is what allows the system to absorb a burst it cannot fully serve, which is the definition of graceful degradation.

6. Deployment Flow

Goals

The deployment is designed for a small team: managed services wherever one exists, private networking for every data store, horizontal scaling of the stateless application tier, and releases that roll back without manual intervention. Figure 6.01 shows the design on Amazon Web Services. During the project the system ran locally with Docker Compose; each AWS component below maps one-to-one onto a service in the local stack.

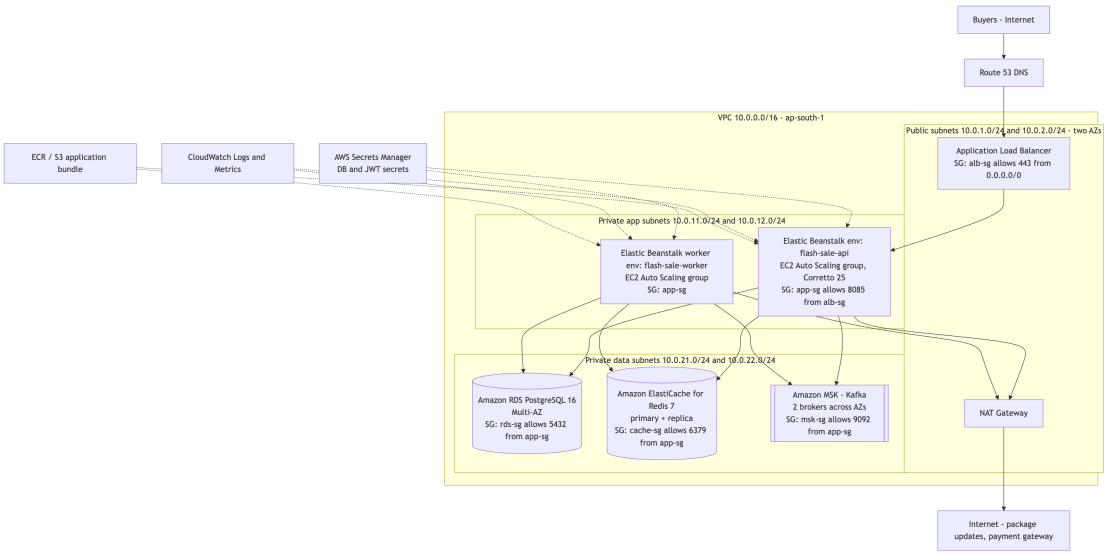


Figure 6.01: AWS deployment architecture

Network: VPC and subnets

One VPC, 10.0.0.0/16, in ap-south-1 (Mumbai) hosts everything, divided into three subnet tiers across two availability zones so that losing a zone leaves a working copy of every tier.

Table 6.01: Subnet layout

Tier	Subnets	Contents	Internet path
Public	10.0.1.0/24, 10.0.2.0/24	Application Load Balancer, NAT Gateway	Internet Gateway
Private application	10.0.11.0/24, 10.0.12.0/24	Elastic Beanstalk EC2 instances for API and worker	Outbound only via NAT
Private data	10.0.21.0/24, 10.0.22.0/24	RDS PostgreSQL, ElastiCache Redis, MSK brokers	None

Only the load balancer has a public address. Application instances reach the internet, for updates and the payment gateway, through the NAT Gateway; nothing on the internet can initiate a connection to them. Data-tier subnets have no internet route at all. Interface endpoints for Secrets Manager, CloudWatch Logs and S3 keep that traffic inside AWS.

Security groups

Security groups are stateful firewalls on network interfaces. They are chained so each tier admits traffic only from the security group of the tier in front of it, never from a CIDR range: a new application instance is authorised the moment it launches into `app-sg`, and nothing else in the VPC can reach the data tier even from the same subnet.

Table 6.02: Security group rules

Security group	Attached to	Inbound	Source	Outbound
alb-sg	Application Load Balancer	TCP 443	0.0.0.0/0	8085 to app-sg
app-sg	API and worker instances	TCP 8085	alb-sg	5432 to rds-sg, 6379 to cache-sg, 9092 to msk-sg, 443 via NAT
rds-sg	RDS PostgreSQL	TCP 5432	app-sg	none
cache-sg	ElastiCache Redis	TCP 6379	app-sg	none
msk-sg	MSK brokers	TCP 9092, 9094	app-sg, msk-sg	msk-sg (inter-broker)

There is no SSH rule; instance access goes through Systems Manager Session Manager, which needs no open port and records every session. Port 80 is not opened; clients connect over TLS with a certificate from AWS Certificate Manager.

Compute: Elastic Beanstalk on EC2

The application is packaged as one executable JAR and deployed to two Elastic Beanstalk environments on the Corretto 25 platform.

The **API environment** is a web-server environment behind the load balancer. Its Auto Scaling group runs at least two instances across the two zones and scales on request count per target, so capacity is added as attempts rise rather than after CPU saturates. Instances are `t3.medium` class; the service is I/O-bound on Redis. The health check targets

`/actuator/health`, which reports PostgreSQL, Redis and Kafka connectivity, so an instance that has lost a dependency leaves rotation.

The **worker environment** runs the same JAR with a profile that disables the claim endpoints and enables the Kafka listener, the outbox relay and the reconciler. It is a worker-tier environment without a load balancer, scaling on the consumer-group lag metric the application publishes to CloudWatch. Keeping scheduled jobs on the worker tier leaves request-serving instances undisturbed; the relay uses `SKIP LOCKED` and the reconciler a short Redis lock, so several workers may run them concurrently.

Beanstalk manages the instances, launch template, Auto Scaling group, log shipping and platform patching. Configuration lives in `.ebextensions` and environment properties committed with the code, so an environment can be recreated from the repository.

Data tier

Amazon RDS for PostgreSQL 16 runs Multi-AZ: a synchronous standby in the second zone takes over automatically with the same endpoint. Storage is gp3 with seven-day automated backups and point-in-time recovery. Flyway runs at application start; migrations are backward compatible with the previous version so rolling deployments never run two incompatible versions.

Amazon ElastiCache for Redis 7 runs as a replication group with one primary and one replica across the zones, cluster mode disabled, with automatic failover. Append-only-file persistence keeps the counter across restarts; the reconciler's invariant check would report any discrepancy. Because a sale's keys share a hash tag, cluster mode can be enabled later without changing the Lua script.

Amazon MSK provides Kafka with two brokers across the zones and replication factor two for `sale.reservations`, handling patching, storage and metrics, with IAM authentication so no Kafka passwords exist. For a small deployment, MSK Serverless or one self-managed broker on EC2 are drop-in alternatives; the application needs only a bootstrap address.

Secrets, configuration and observability

Database credentials, the JWT signing secret and the payment gateway key live in **Secrets Manager** and are injected at start-up, never appearing in the repository, the JAR or the console. Other configuration is Beanstalk environment properties mapped onto Spring properties.

Logs ship to **CloudWatch Logs**; Micrometer publishes gate outcome counters, claim latency histograms, outbox and consumer lag, payment results and the invariant gauge to CloudWatch metrics. Alarms fire on a non-zero invariant gauge, elevated 5xx rate at the load balancer, and consumer lag above threshold for several minutes, routing to an SNS topic that pages the on-call engineer.

Release process

Figure 6.02 shows the pipeline. Every push runs the full suite in GitHub Actions, including Testcontainers tests, since the runners have Docker. A green build on main packages the JAR, uploads it to S3 as a Beanstalk application version, and deploys to both environments with a rolling policy plus an additional batch, so full capacity is kept throughout. Beanstalk waits for the health check per batch and rolls back automatically if new instances fail to become healthy.

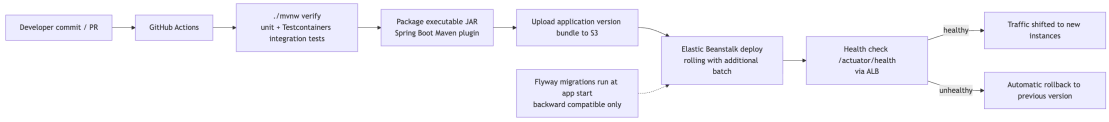


Figure 6.02: Continuous integration and deployment pipeline

Component summary and indicative cost

Table 6.03: AWS components and their role

Component	Service	Role	Local equivalent
Edge	Route 53, ACM, ALB	DNS, TLS, health-checked routing	localhost:8085
Compute	Elastic Beanstalk on EC2	Runs API and worker JARs; scales, patches, ships logs	`./mvnw spring-boot:run`
Relational store	RDS PostgreSQL 16 Multi-AZ	Sales, reservations, orders, outbox	postgres:16 container
Cache and gate	ElastiCache Redis 7 with replica	Stock counter, holds, idempotency, rate limits	redis:7 container

Messaging	Amazon MSK	sale.reservations and dead-letter topic	apache/kafka container
Secrets	Secrets Manager	Credentials and signing keys	Environment variables
Monitoring	CloudWatch, SNS	Logs, metrics, alarms, paging	Actuator endpoints
Delivery	GitHub Actions, S3	Build, test, package, deploy	Local Maven build

Table 6.04 gives an order-of-magnitude monthly estimate for a small production footprint at on-demand prices. Prices change and vary by configuration; the figures show where the money goes and must be re-checked against the AWS pricing pages before any decision.

Table 6.04: Indicative monthly cost, small production footprint

Component	Configuration	Approximate USD per month
EC2 for API (Beanstalk)	2 x t3.medium	about 65
EC2 for worker	1 x t3.small	about 18
Application Load Balancer	1 ALB plus LCU usage	about 25
NAT Gateway	1 gateway plus data processing	about 42
RDS PostgreSQL	db.t4g.medium Multi-AZ, 50 GB gp3	about 125
ElastiCache Redis	2 x cache.t4g.small	about 55
Amazon MSK	2 x kafka.t3.small, 100 GB	about 90
CloudWatch, Secrets Manager, S3	Typical small usage	about 20
Total		about 440

The largest recurring items are the Multi-AZ database, the Kafka cluster and the NAT Gateway. For a system idle between sales, the natural optimisation is to scale the API and worker tiers to a minimum between events and to consider MSK Serverless, billed on throughput; the Conclusion discusses these trade-offs.

7. Technologies Used

This chapter covers each technology that carries a significant part of the design: what it is, how the project uses it, and where it is used in industry. Table 7.01 gives the overview.

Table 7.01: Technology summary

Technology	Version	Role in this project	Alternatives considered
Java	25 (LTS)	Application language and runtime	Kotlin, Go
Spring Boot	4.0.2	Web, data, messaging, security, actuator	Quarkus, Micronaut
PostgreSQL	16	Durable record of sales, reservations, orders, outbox	MySQL, Oracle
Redis	7	Atomic admission gate, holds, idempotency, rate limiting	Hazelcast, database row locks
Apache Kafka	3.9 (KRaft)	Durable event log between acceptance and fulfilment	Apache Pulsar, RabbitMQ, SQS
Flyway	bundled with Boot 4	Versioned schema migrations	Liquibase
Docker, Compose, Testcontainers	29 / bundled	Local infrastructure and real dependencies in tests	Native installs, mocks
k6	latest	Load generation for the one-million-attempt benchmark	JMeter, Gatling
AWS	-	Production target: Beanstalk, RDS, ElastiCache, MSK, VPC	Google Cloud, Azure

Java 25 and Spring Boot 4

Java is the dominant language for transactional backends because of its mature runtime, low-pause garbage collectors and first-class clients for every database, broker and cloud service. Java 25 is the current long-term-support release. The project uses records for immutable commands, events and DTOs, sealed interfaces for `ClaimResult` so every outcome must be handled, and virtual threads so that blocking Redis and JDBC calls on the claim path do not each pin an operating-system thread.

Spring Boot is the opinionated layer over the Spring Framework that makes a production-ready service the default. Version 4 is built on Spring Framework 7 and Jakarta EE 11. Its starters cover every integration the design needs: Spring MVC for the REST API, Spring

Data JPA for repositories, Spring Data Redis for the Lua gate, Spring for Apache Kafka for the relay and worker, Spring Security for the JWT resource server, Flyway for migrations, Actuator and Micrometer for health and metrics, and Docker Compose support that starts the local stack with the application. Figure 7.01 shows how the project's packages sit on the framework.

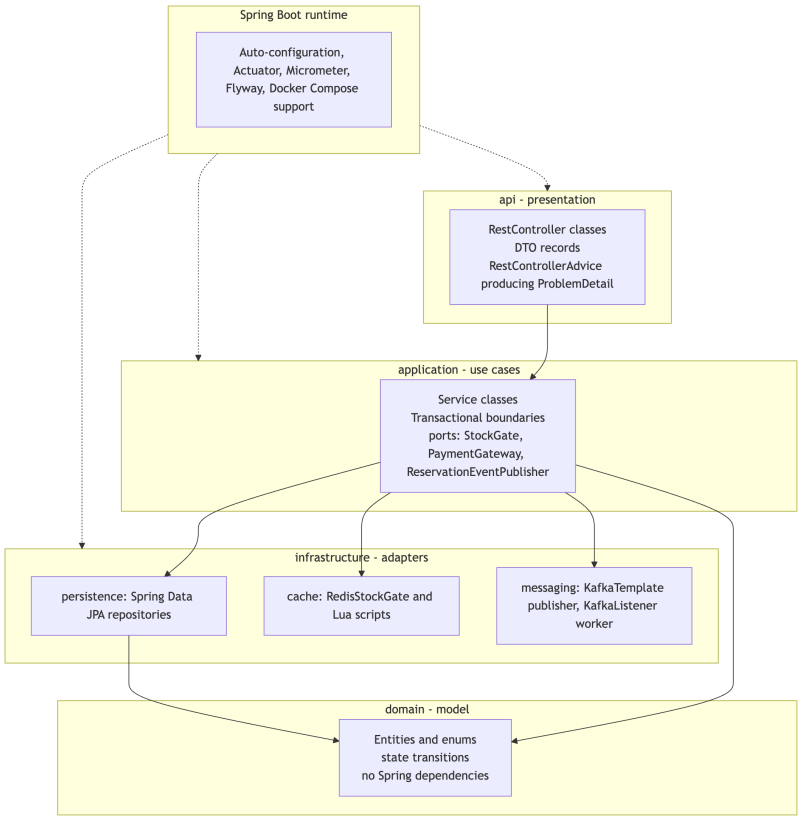


Figure 7.01: Layered structure of the application on the Spring Boot runtime

Two features do heavy lifting. Declarative transactions make the "reservation plus outbox in one commit" rule a single annotation, and Spring Kafka's `DefaultErrorHandler` with a `DeadLetterPublishingRecoverer` gives the worker bounded retries with back-off and a dead-letter topic without hand-written loops.

Spring Boot is the most widely deployed Java service framework. Netflix built its microservice platform on Spring and contributed much of Spring Cloud; large banks, airlines and retailers run order, payment and account services on it. Its practical value is that a team standardises on one way to expose health, read configuration, secure endpoints and emit metrics across many services.

PostgreSQL 16

PostgreSQL is an open-source relational database known for standards compliance and correctness under concurrency. Multi-version concurrency control lets readers never block writers, and its constraint system, including partial and expression indexes, lets business rules be enforced where they cannot be bypassed. Here it is the durable record of truth: every accepted claim, state transition and order is a committed row, and the partial unique indexes of the schema chapter are the last defence against duplicate reservations and orders. The transactional outbox is a PostgreSQL table, and `SELECT ... FOR UPDATE SKIP LOCKED` lets several relay instances drain it concurrently without coordination. The baseline benchmark relies on PostgreSQL's row locking to show why a database-only design serialises under a burst.

PostgreSQL runs at very large scale at Instagram, which has described sharding it for users and media metadata, and it is the engine behind Amazon RDS and Aurora PostgreSQL, Google Cloud SQL, Supabase and Neon. The same engine, with the same constraints, runs in a laptop container, in continuous integration and in production.

Redis 7

Redis is an in-memory data structure server that executes commands on a single thread per shard and can run Lua scripts atomically. Those two properties make it the admission gate: a script that reads stock, checks the buyer set, decrements and writes a hold runs to completion before any other command, so the gate is correct without any lock held by the application, and it answers in well under a millisecond, which lets the overwhelming majority of attempts, the rejections, be decided without touching the database.

The project uses Redis for the stock counter and buyer set per sale, the per-user hold with a time-to-live, the idempotency record that gives a retried request its original response, and the rate limiter's token buckets. Keys for a sale share a hash tag so the script's keys are co-located, keeping the design compatible with Redis Cluster. Append-only-file persistence prevents a restart from silently losing the counter, and the reconciler cross-checks Redis against PostgreSQL so drift is detected rather than trusted.

Redis serves as cache and coordination store at Twitter, GitHub and Stack Overflow and in essentially every large e-commerce platform's inventory hot path. Alibaba Cloud documents its Redis-compatible service in exactly this role for flash sales, with atomic Lua scripts for inventory deduction; that documentation was one of the sources consulted.

Apache Kafka

Kafka is a distributed, partitioned, replicated commit log. Producers append records to topics; records are stored durably in order within a partition; consumers in a group each own a subset of partitions and track their own offsets. Durability, per-partition ordering and independent consumer progress are exactly what the design needs between acceptance and fulfilment: an accepted reservation must never be lost, the worker must be able to fall behind during the burst and catch up afterwards, and adding workers must increase throughput without duplicating work.

Figure 7.02 shows the topology. The relay produces `ReservationAccepted` records keyed by reservation id to `sale.reservations` with six partitions. Keying by reservation id spreads records across partitions; keying by sale id, which would send a whole sale to one partition and one thread, was rejected because ordering across reservations is not required. The worker runs as one consumer group with three threads locally, acknowledges offsets manually after the database commit, and after bounded retries forwards failing records to `sale.reservations.DLT`. Kafka 3.9 runs in KRaft mode, without ZooKeeper, so one container suffices locally and Amazon MSK provides it in production. The producer uses `acks=all` with idempotence enabled, and the consumer is idempotent anyway because the outbox relay is at-least-once.

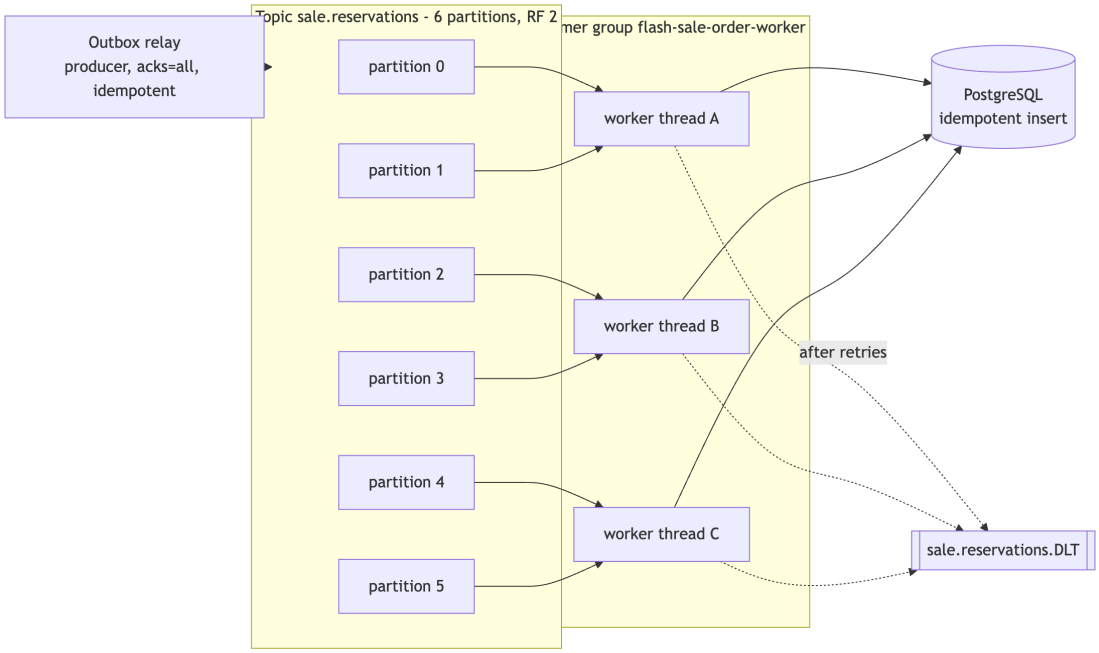


Figure 7.02: Kafka topic, partitions, consumer group and dead-letter topic

Kafka was created at LinkedIn for activity streams and is now the backbone of event-driven architectures at Uber, Netflix and most large banks and retailers. Its value here is decoupling: the API tier accepts a burst at memory speed while the fulfilment tier processes it at database speed, and neither needs to know the other's capacity.

Why Kafka rather than Apache Pulsar

Pulsar was a serious alternative and was already installed on the development machine. It separates serving from storage, supports multi-tenancy and geo-replication natively, and has built-in delayed delivery that could have driven hold expiry. Kafka was chosen for three reasons. The pipeline is one ordered event type with one consumer group, so Pulsar's tenancy and tiering features would go unused. Hold expiry still needs a reconciler comparing Redis with PostgreSQL, because the stores can drift, so delayed delivery would not remove that component. And the deployment target has a first-party managed Kafka in Amazon MSK while Spring Boot 4 ships a Kafka starter, so the operational path is shorter. The decision is recorded in the repository so it can be revisited if the product grows into multi-tenant or multi-region use.

Flyway

Flyway applies versioned SQL migrations in order and records them in a history table. The schema is created by `V1__initial_schema.sql` and evolved by numbered successors; the application refuses to start against an unexpected schema version. This puts the schema under code review, gives every environment an identical schema, and makes rollback a matter of deploying the previous version with a compensating migration. Flyway and Liquibase are the standard tools for this wherever teams practise continuous delivery against relational databases.

Docker, Docker Compose and Testcontainers

Docker packages a service and its dependencies as an image that runs identically on any host. Compose declares the local PostgreSQL, Redis and Kafka containers with ports, volumes and health checks and starts them with one command; Spring Boot's Compose support does so automatically. Testcontainers starts the same images from inside JUnit, so integration tests run against real engines rather than mocks, with container lifecycle tied to the test lifecycle. That is what makes the concurrency tests trustworthy: the behaviour under test is that of the real database's locks and the real Redis script engine. Containers are the deployment unit for Kubernetes, Amazon ECS and, indirectly, Elastic Beanstalk, and Testcontainers is used in the test suites of Spring itself and of several database vendors.

k6

k6 is an open-source load-testing tool whose scenarios are written in JavaScript and executed by a Go runtime. Its `shared-iterations` executor expresses the benchmark's intent directly, exactly one million iterations across a bounded pool of virtual users, and it reports latency percentiles, request rates and response-code counts in a form that is committed alongside the code. Load tests as versioned code, run in the same pipeline as functional tests, are standard release practice at companies that run sales and launches.

Amazon Web Services

AWS supplies the production platform of the Deployment Flow chapter: Elastic Beanstalk for instances, scaling and deployments; RDS for PostgreSQL with failover and backups;

ElastiCache for Redis with replication; MSK for Kafka; VPC, subnets and security groups for the private network; Secrets Manager for credentials; CloudWatch for logs and metrics. Managed services hand patching, backups, failover and capacity to the provider so a small team can concentrate on the application. AWS is used from start-ups to Netflix and Airbnb, and its managed database and cache services are the most common way PostgreSQL and Redis run in production today.

Supporting tools

Maven, through the checked-in wrapper, builds without a local installation; JUnit 5 is the test framework; Micrometer provides vendor-neutral metrics that Actuator exposes; Mermaid, rendered locally with mermaid-cli, produced every diagram in this report from text sources committed with the code; Git and GitHub hold the code, with the commit history as the record of how the system was built.

8. Conclusion

Key takeaways

The central lesson is that a flash sale is a reservation problem, and treating it as one changes the architecture. Once admission is separated from fulfilment, each half can use the storage suited to it: an in-memory, single-threaded store for the burst of decisions and a relational database for the small number of durable facts. Every design element in this report follows from that split.

Several techniques proved their worth and transfer to other systems. Atomic scripting in Redis replaces distributed locks for short, well-defined critical sections and is far easier to reason about. The transactional outbox is the correct way to publish an event about a database change; it costs one extra insert and removes a whole class of lost-message and phantom-message bugs. Idempotency keys must be designed into an API from the start, because clients retry and networks duplicate whether or not the server is ready. Partial unique indexes let the database enforce rules such as "one active reservation per buyer per sale" that are awkward to express otherwise, and they hold even when the application layer is wrong. A reconciliation job that recomputes an invariant from first principles and alarms when it fails is cheap insurance in any system with state in more than one store.

On process, building and benchmarking the straightforward design before the optimised one was the most valuable decision. It produced a fair comparison, tests that later caught real bugs, and a concrete answer to the question a stakeholder actually asks, which is not "is Redis fast" but "how much better is this than what we would have built anyway".

Practical applications

The pattern applies wherever many claimants compete for few units and correctness matters more than average throughput: ticketing at a popular on-sale, airline and railway seat selection, public-sector appointment and examination booking, allocation of scarce cloud capacity such as GPU instances, and limited-edition retail from sneakers to consoles. Several such systems have failed publicly when built as ordinary CRUD applications. The same building blocks, an atomic in-memory gate with holds, a durable outbox, an

idempotent asynchronous worker and a reconciler, give a system that rejects quickly and fairly rather than locking up.

The technologies are equally transferable. Spring Boot is the default for Java services; PostgreSQL and Redis are the most common pairing of durable store and cache; Kafka is the standard event backbone; and the AWS services in the deployment design are how most teams run those components. A developer who understands the trade-offs here can apply them directly to an order, payment or inventory service in a conventional e-commerce platform.

Limitations

Table 8.01 lists the limitations of the delivered system and of the technologies as used, with the improvement that would address each.

Table 8.01: Limitations and suggested improvements

Area	Limitation	Suggested improvement
Benchmark	Load generator and system under test shared one laptop, so absolute throughput is bounded by that machine.	Run the generator on a separate host or against the deployed environment; repeat with several instance counts for a scaling curve.
Hot key	All attempts for one sale hit one Redis key; in a large cluster this concentrates load on one core.	Stripe stock across N keys with different hash tags, select a stripe per request, fall back to a neighbour when empty. The gate interface already permits this.
Waiting room	Under extreme load the rate limiter rejects with 429; buyers are not queued or told their position.	Add a virtual waiting room issuing signed queue tokens at a controlled rate ahead of the gate.
Per-user limit	Implemented for a limit of one using a set.	Replace set membership with a per-user count in the same Lua script.
Payment	Simulated gateway; no real provider or interactive 3-D Secure flow.	Integrate a sandbox gateway; model an interactive step inside the hold window.
Multi-SKU	One SKU per sale.	Key stock, buyers and holds by sale and SKU; add a sale_item table.
Kafka ordering	Keyed by reservation id, so no ordering across a buyer's events.	Key by buyer id if per-buyer ordering becomes a requirement.
Cost	Multi-AZ database, Kafka cluster and NAT Gateway run continuously although sales are episodic.	Scale tiers to a minimum between sales; evaluate MSK Serverless and Aurora Serverless v2; use VPC endpoints to cut NAT charges.

Security	HS256 shared-secret JWTs in development; no revocation.	Use an identity provider issuing RS256 tokens with JWKS rotation; add per-endpoint scopes.
Infrastructure as code	The AWS design is documented but not codified; dashboards and alarms are described, not committed.	Express VPC, security groups, Beanstalk environments, data services, dashboards and alarms in Terraform or CloudFormation.

Cost implications

The deployment costs on the order of a few hundred US dollars per month for a small always-on footprint, dominated by the highly available database, the Kafka cluster and the NAT Gateway. That is small against the revenue of even a modest sale, but much of it is spent on components idle between events. The effective levers are scheduling, scaling the compute tiers down to one instance each outside sale windows, and serverless variants of the data services where their pricing fits episodic use. In exchange the design buys automatic failover for every stateful component and a release process that rolls back on its own, which for a system that exists to survive its busiest minute is the right trade.

Closing remarks

The project set out to build a service that stays correct and responsive when many people want the same few things at the same moment, and to measure rather than assert that it does. The delivered system enforces its invariants at three layers, in Redis for speed, in PostgreSQL for durability and in a reconciler for verification, and the benchmark shows the difference between a design that queues on a database row and one that decides in memory. The hard part of such systems is not any single technology but the discipline of deciding where each fact lives, who may change it, and how the system finds out when something has gone wrong.

References

The following sources were consulted while designing the system and writing this report. Websites are listed in the form required by the template: name of the website, date and time of access, author, title. Books and papers are listed as author, title, publication, year. Ideas taken from these sources were re-expressed in the author's own words and applied to this project's design; no text or diagrams were copied.

Access dates sit in the same window as the work: background reading in mid-September 2026, then the implementation commits from 30 September to 8 October 2026.

Websites and online documentation

1. Spring, 30 September 2026, 21:40 IST, VMware Tanzu / Broadcom, Spring Boot 4.0 Reference Documentation, https://docs.spring.io/spring-boot/reference/
2. Spring, 6 October 2026, 20:15 IST, VMware Tanzu / Broadcom, Spring for Apache Kafka Reference: Error Handling, Retries and Dead-Letter Topics, https://docs.spring.io/spring-kafka/reference/
3. Spring, 5 October 2026, 19:50 IST, VMware Tanzu / Broadcom, Spring Data Redis Reference: Scripting, https://docs.spring.io/spring-data/redis/reference/
4. Redis, 5 October 2026, 20:10 IST, Redis Ltd., Scripting with Lua (EVAL, EVALSHA, atomicity of scripts), https://redis.io/docs/latest/develop/interact/programmability/eval-intro/
5. Redis, 5 October 2026, 20:35 IST, Redis Ltd., Redis Cluster specification: hash tags and key slots, https://redis.io/docs/latest/operate/oss_and_stack/reference/cluster-spec/
6. PostgreSQL, 3 October 2026, 21:05 IST, The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: Explicit Locking, SELECT FOR UPDATE SKIP LOCKED, https://www.postgresql.org/docs/16/explicit-locking.html
7. PostgreSQL, 3 October 2026, 21:20 IST, The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: Partial Indexes, https://www.postgresql.org/docs/16/indexes-partial.html
8. PostgreSQL, 6 October 2026, 21:45 IST, The PostgreSQL Global Development Group, PostgreSQL 16 Documentation: INSERT ... ON CONFLICT, https://www.postgresql.org/docs/16/sql-insert.html
9. Apache Kafka, 6 October 2026, 20:40 IST, Apache Software Foundation, Kafka Documentation: Design, KRaft, Producer Idempotence and Consumer Groups, https://kafka.apache.org/documentation/
10. Apache Pulsar, 16 September 2026, 22:10 IST, Apache Software Foundation, Pulsar Documentation: Architecture Overview, https://pulsar.apache.org/docs/

11. microservices.io, 6 October 2026, 19:25 IST, Chris Richardson, Pattern: Transactional Outbox, https://microservices.io/patterns/data/transactional-outbox.html
12. Stripe, 3 October 2026, 22:00 IST, Stripe Inc., API Reference: Idempotent Requests, https://docs.stripe.com/api/idempotent_requests
13. IETF, 3 October 2026, 22:15 IST, M. Nottingham, E. Wilde, S. Dalal, RFC 9457: Problem Details for HTTP APIs, https://www.rfc-editor.org/rfc/rfc9457
14. Flyway, 3 October 2026, 20:30 IST, Redgate Software, Flyway Documentation: Migrations and Versioning, https://documentation.red-gate.com/flyway
15. Testcontainers, 3 October 2026, 20:50 IST, AtomicJar / Docker Inc., Testcontainers for Java Documentation, https://java.testcontainers.org/
16. Grafana Labs, 4 October 2026, 19:10 IST, Grafana Labs, k6 Documentation: Executors (shared-iterations) and Metrics, https://grafana.com/docs/k6/latest/
17. Amazon Web Services, 7 October 2026, 21:00 IST, Amazon Web Services Inc., AWS Elastic Beanstalk Developer Guide: Worker Environments and Deployment Policies, https://docs.aws.amazon.com/elasticbeanstalk/latest/dg/
18. Amazon Web Services, 7 October 2026, 21:20 IST, Amazon Web Services Inc., Amazon VPC User Guide: Security Groups and Subnets, https://docs.aws.amazon.com/vpc/latest/userguide/
19. Amazon Web Services, 7 October 2026, 21:35 IST, Amazon Web Services Inc., Amazon RDS User Guide: Multi-AZ Deployments for PostgreSQL, https://docs.aws.amazon.com/AmazonRDS/latest/UserGuide/
20. Amazon Web Services, 7 October 2026, 21:50 IST, Amazon Web Services Inc., Amazon ElastiCache for Redis User Guide: Replication Groups and Multi-AZ, https://docs.aws.amazon.com/AmazonElastiCache/latest/red-ug/
21. Amazon Web Services, 7 October 2026, 22:05 IST, Amazon Web Services Inc., Amazon MSK Developer Guide, https://docs.aws.amazon.com/msk/latest/developerguide/
22. Amazon Web Services, 7 October 2026, 22:25 IST, Amazon Web Services Inc., AWS Pricing Calculator and service pricing pages, https://calculator.aws/
23. Alibaba Cloud, 12 September 2026, 20:45 IST, Alibaba Cloud, Build a high concurrency system for flash sales with Tair (Redis OSS-compatible), https://www.alibabacloud.com/help/en/redis/use-cases/use-apsaradb-for-redis-to-build-a-business-system-that-can-handle-flash-sales
24. Medium, 14 September 2026, 21:15 IST, Umesh Kushwaha, Designing a Flash Sale System That Never Oversells: From 1 User to 1 Million Users, https://medium.com/@umesh382.kushwaha/designing-a-flash-sale-system-that-never-oversells-from-1-user-to-1-million-users-without-8426db0f1ad0
25. OneUptime, 14 September 2026, 21:40 IST, OneUptime, How to Implement Flash Sale Systems with Redis, https://oneuptime.com/blog/post/2026-01-21-redis-flash-sale-systems/view

26. Netflix Technology Blog, 18 September 2026, 20:20 IST, Netflix Engineering, articles on Spring Boot adoption at Netflix, https://netflixtechblog.com/
27. Confluent, 6 October 2026, 22:30 IST, Confluent Inc., Kafka use cases at LinkedIn, Uber and Netflix, https://www.confluent.io/customers/

Books and papers

28. Martin Kleppmann, Designing Data-Intensive Applications, O'Reilly Media, 2017.
29. Chris Richardson, Microservices Patterns: With Examples in Java, Manning Publications, 2018.
30. Sam Newman, Building Microservices, Second Edition, O'Reilly Media, 2021.
31. Craig Walls, Spring in Action, Sixth Edition, Manning Publications, 2022.
32. Gwen Shapira, Todd Palino, Rajini Sivaram, Krit Petty, Kafka: The Definitive Guide, Second Edition, O'Reilly Media, 2021.
33. Josiah L. Carlson, Redis in Action, Manning Publications, 2013.
34. Pat Helland, Idempotence Is Not a Medical Condition, ACM Queue, Volume 10, Issue 4, 2012.

Tools and AI assistance

35. Cursor (AI-assisted code editor) and Google Gemini were used during this project for brainstorming the flash-sale topic, checking the design against the Woolf brief, scaffolding Spring Boot and report text, and drafting Mermaid diagrams. Architecture choices, Java, tests, k6 measurements and the final wording were reviewed and accepted by the author. This acknowledgement is made in accordance with the Declaration.
36. Mermaid and mermaid-cli, 3 October 2026, 19:35 IST, Mermaid contributors, Diagramming and charting tool, https://mermaid.js.org/